Resumen Unidad 2

Creado	@15 de enero de 2026 22:41
Etiquetas	

BBDD Orientadas a Documentos: MongoDB

Este tipo de base de datos sigue un modelado parecido a JSON, estando conformado como su nombre indica por documentos, que no son mas que conjuntos de datos pertenecientes a una unidad.

El modelado de datos es mucho mas accesible para aplicaciones que el empleado por las bbdd relacionales con la posibilidad de agrupar o anidar dentro de un documento. Lo que proporciona un acceso eficiente que a su vez se realiza por claves con un identificador y variables indexadas.

Requisitos de rendimiento

- Para garantizar el rendimiento debemos utilizar indices en variables utilizadas en filtro.
- Evitar y/o reducir el uso de join ($lookup) y anidar la información de manera coherente
- Utilizar indexado inverso cuando se realicen consultas sobre el texto o expresiones

Indicios de mala praxis y problemas de diseño:

- BBDD con multiples colecciones
- Arrays de muchos elementos
- Cantidad de informacion por documento excesiva
- Gran cantidad de indices

Concepto CRUD

CRUD es un acrónimo que define las operaciones básicas sobre bases de datos:

- **C – Create (Crear)**: Insertar nuevos documentos.
- **R – Read (Leer)**: Consultar o recuperar documentos existentes.
- **U – Update (Actualizar)**: Modificar documentos existentes.

- **D – Delete (Eliminar)**: Borrar documentos existentes.

Índices

Los índices permiten optimizar las consultas y ordenar los datos de manera eficiente.

- **createIndex()**: Crea un índice en el campo especificado si aún no existe.
- **dropIndex()**: Elimina un índice existente.
- **getIndexes()**: Obtiene los índices de una colección.
- **Tipos de índices**:
  - Ascendente (`1`) / Descendente (`1`)
  - Texto (`text`)
  - Geoespacial: `2d` y `2dsphere`

```
db.collection.createIndex(
{
   "a": 1
}
{
   unique: true,
   sparse: true,
   expireAfterSeconds: 3600
}
)
```

CRUD en MongoDB

Lectura (Read)

- **find()**: Devuelve un cursor con los documentos que cumplen los criterios.
- **findOne()**: Devuelve un único documento que cumple los criterios.
- **Sintaxis**:

```
db.collection.find(<query>, <projection>, <options>)
db.collection.findOne(<query>, <projection>, <options>)
```

3.2 Escritura (Create)

- **insertOne()**: Inserta un documento.
- **insertMany()**: Inserta varios documentos.
- **Sintaxis**:

```
db.collection.insertOne({item: "card", qty: 15});
```

```
db.collection.insertMany([{item: "card", qty: 15}, {item: "stamps", qty: 50}])
```

- Devuelve la *id* del documento insertado o la lista de *ids*.

3.3 Actualización (Update)

- **updateOne() / updateMany()**: Actualiza uno o varios documentos según un filtro.
- **replaceOne()**: Reemplaza un documento completo.
- **Opcional** `upsert`: Si no existe el documento, lo inserta.
- **Sintaxis**:

```
db.collection.updateOne({"name" : "Central Cafe"}, {$seet: {violations : 3}})
db.collection.updateMany(<filter>, <update>, <options>)
db.collection.replaceOne(<filter>, <replacement>, <options>)
```

- **findAndModify()**: Modifica o elimina y devuelve un único documento.
  - Por defecto devuelve el documento original.
  - Para devolver el documento modificado: `new: true`.

```
db.collection.findAndModify({
  query: {name: "Andy" },
  update: { $inc: { score: 1 }},
    upsert: true
});
```

3.4 Eliminación (Delete)

- **deleteOne()**: Elimina un único documento que cumple el filtro.
- **deleteMany()**: Elimina todos los documentos que cumplen el filtro.
- **Sintaxis**:

```
db.collection.deleteOne(<filter>)
db.collection.deleteMany(<filter>)
```

```
db.orders.deleteOne(
        {_id: ObjectId("54765465245235da")},
        {w: "majority", wtimeout: 100}
);
```

---

4. Operadores de búsqueda (Filtros)

4.1 Comparación

- `$gt` : mayor que

- `$gte` : mayor o igual que
- `$lt` : menor que
- `$lte` : menor o igual que
- `$ne` : diferente a
- `$in` : dentro de una lista
- `$nin` : no dentro de una lista

```
db.inventory.find({ quantity: { $gt: 20}})
db.inventory.find( {quantity: { $in: [5, 15]}}, {_id: 0})
```

4.2 Lógicos

- `$and` : y lógico
- `$or` : o lógico
- `$not` : negación
- `$nor` : negación de OR

```
db.inventory.find( {
   $and: [
         { $or: [ { qty: { $lt : 10 }}, { qty : { $gt: 50 } } ] },
         { $or: [ sale: true }, { price : { $lt : 5 }} ] }
         ]
} )
```

4.3 Elemento

- `$exists` : el campo existe
- `$type` : el campo es de un tipo determinado

```
db.spices.find( { saffron: { $exits: true } } )
```

4.4 Evaluación

- `$mod:` Realiza el módulo del valor de un campo y comprueba si el resultado coincide con el especificado.
- `$regex` **:** Permite realizar búsquedas mediante expresiones regulares.

  Requiere indexado; en MongoDB Atlas se recomienda usar **Full-Text Search**.
- `$text` **:** Búsqueda de texto sobre índices de tipo `text`.

  Requiere indexado; en Atlas se utiliza **Full-Text Search**.
- `$where` **:** Permite realizar búsquedas utilizando expresiones JavaScript (uso desaconsejado por rendimiento y seguridad).

```
db.articles.find( { $text: { $search: "coffee" } } )
```

4.5 Operadores geoespaciales

- `$geoIntersects:` Devuelve documentos cuya geometría intersecta con una geometría especificada. Requiere índice `2dsphere`.
- `$geoWithin` **:** Devuelve documentos que se encuentran dentro de una geometría especificada. Compatible con índices `2d` y `2dsphere`.
- `$near` **:** Devuelve documentos ordenados por distancia cartesiana a un punto.
- `$nearSphere` **:** Devuelve documentos ordenados por distancia geodésica (esférica).
- `$minDistance` **/** `$maxDistance` **:** Limita el rango de distancia en consultas de proximidad.
- **GeoJSON:** Formato estándar para definir geometrías geográficas (longitud, latitud).

```
db.restaurantes.find({
  ubicacion: {
    $nearSphere: {
      $geometry: {
        type: "Point",
        coordinates: [ -3.7040, 40.4170 ]
      },
      $maxDistance: 1000
    }
  }
})
```

---

5. Operadores sobre arrays (búsqueda)

- **Consulta normal:** Devuelve documentos cuyo array contiene al menos un elemento que cumple la condición.
- `$elemMatch` **:** Devuelve documentos cuyo array contiene al menos un elemento que cumple múltiples condiciones.
- `$all` **:** Comprueba que todos los elementos de una lista estén contenidos en el array.
- `$size` **:** Devuelve documentos cuyo array tiene un tamaño exacto.

```
db.scores.find (
    { results: { $elemMatch: { $gte: 80, $lt: 85 } } }
)
```

CRUD: Proyección en arrays

1. Limitación del resultado en arrays

- `$` **:** Devuelve únicamente la primera coincidencia del array que cumple la condición de búsqueda.
- `$elemMatch` **(en proyección):** Devuelve únicamente el primer elemento del array que cumple una condición especificada. Puede devolver el array vacío.
- `$slice` **:** Devuelve solo los elementos del array comprendidos entre los índices especificados.

```
db.students.find ( { grades: { $elemMatch: {
                                mean: { $gt: 70},
                                grade: { $gt: 90 }
                                } } },
                    { "grades.$": 1 } )
```

CRUD: Operadores de modificación

2. Operadores sobre campos

- `$currentDate` **:** Inserta la fecha actual en el campo.
- `$inc` **:** Incrementa el valor de un campo según el valor indicado.
- `$max` **:** Actualiza el campo solo si el valor especificado es mayor.
- `$min` **:** Actualiza el campo solo si el valor especificado es menor.
- `$mul` **:** Multiplica el valor del campo por el valor indicado.
- `$rename` **:** Cambia el nombre de un campo.
- `$setOnInsert` **:** Inicializa un campo solo si el documento se inserta.
- `$set` **:** Crea o actualiza el valor de un campo.
- `$unset` **:** Elimina un campo del documento.

```
db.products.updateOne(
{ _id: 1 },
{
   $set: { item: "apple"},
   $setOnInsert: { defaultQty: 100 }
},
   {upsert: true }
}
```

---

3. Operadores sobre arrays (modificación)

- `$` **:** Aplica la modificación solo a la primera coincidencia del array.
- `$[]` **:** Aplica la modificación a todos los elementos del array.
- `$addToSet` **:** Añade un elemento al array solo si no existe previamente.
- `$pop` **:** Elimina el primer o el último elemento del array.
- `$pullAll` **:** Elimina todos los elementos del array que coincidan con una lista dada.

- `$pull` : Elimina los elementos del array que cumplan una condición.
- `$push` : Añade un elemento al array.

```
db.students.updateOne(
   {_id: 1, grades: 80},
   {$set: { "grades.$" : 82 } }
)
```

---

4. Modificadores de arrays

- `$each` : Inserta múltiples elementos en un array. Se utiliza junto con `$push` o `$addToSet`.
- `$position` : Inserta los elementos en una posición específica del array (`$push + $each`).
- `$slice` : Limita el tamaño del array durante la inserción (`$push + $each`).
- `$sort` : Ordena los elementos del array durante la inserción (`$push + $each`).

```
db.students.updateOne(
   {_id: 2},
   {$push : {tests: {$each: [40, 60], $sort: 1}} }
)
```

Consultas agregadas en MongoDB

Pipeline de agregación

- Una consulta agregada se define como una **secuencia de etapas (stages)**.
- Cada etapa transforma los documentos y pasa el resultado a la siguiente.
- El resultado final es un **listado de documentos**.

Sintaxis general:

```
db.collection.aggregate( <pipeline>, <options> )
```

Ejemplo básico:

```
db.users.aggregate([
  { $match: { age: { $gte: 18 } } },
  { $project: { name: 1, age: 1 } }
])
```

Stages principales del pipeline

`$match`

- Filtra documentos según una condición.
- Usa la **misma sintaxis que** `find()`.

- Conviene colocarlo al inicio del pipeline para mejorar rendimiento.

Sintaxis:

```
{ $match: { <query predicate> } }
```

Ejemplo:

```
{ $match: { status: "active", age: { $gte: 18 } } }
```

$lookup

- Realiza una combinación entre colecciones (JOIN).
- El resultado del join se guarda en un **array**.

Sintaxis:

```
{
  $lookup: {
    from: "orders",
    localField: "_id",
    foreignField: "userId",
    as: "orders"
  }
}
```

Ejemplo:

```
db.users.aggregate([
  {
    $lookup: {
      from: "orders",
      localField: "_id",
      foreignField: "userId",
      as: "orders"
    }
  }
])
```

$group

- Agrupa documentos según una clave (`_id`).
- Permite usar **acumuladores** para calcular valores.

Sintaxis:

```
{
  $group: {
    _id: <expression>,
```

```
    <field>: { <accumulator>: <expression> }
  }
}
```

Ejemplo:

```
{
  $group: {
    _id: "$category",
    total: { $sum: "$price" },
    avgPrice: { $avg: "$price" }
  }
}
```

Operadores acumuladores

- `$sum`: suma valores
- `$avg`: media
- `$first`: primer valor del grupo
- `$last`: último valor del grupo
- `$max`: valor máximo
- `$min`: valor mínimo
- `$push`: crea un array con todos los valores
- `$addToSet`: crea un array sin duplicados

Ejemplo:

```
{
  $group: {
    _id: "$department",
    employees: { $addToSet: "$name" }
  }
}
```

`$unwind`

- Descompone un array creando un documento por elemento.

Sintaxis:

```
{ $unwind: { path: "$items" } }
```

Ejemplo:

```
{ $unwind: "$orders" }
```

`$sort`

- Ordena documentos.

Sintaxis:

```
{ $sort: { field: 1 | -1 } }
```

Ejemplo:

```
{ $sort: { age: -1 } }
```

`$limit`

- Limita el número de documentos devueltos.

Sintaxis:

```
{ $limit: 5 }
```

`$skip`

- Omite los primeros documentos.

Sintaxis:

```
{ $skip: 10 }
```

`$geoNear`

- Ordena documentos por cercanía a un punto geográfico.
- Debe ser **la primera etapa del pipeline**.
- Requiere índice `2dsphere`.

Sintaxis:

```
{
  $geoNear: {
    near: { type: "Point", coordinates: [lon, lat] },
    distanceField: "dist",
    spherical: true
  }
}
```

`$out`

- Escribe el resultado en una colección.
- Debe ser la **última etapa**.
- Si la colección existe, se sobrescribe.

Sintaxis:

```
{ $out: "result_collection" }
```

`$project`

- Remodela documentos.
- Permite incluir, excluir o crear campos.
- `_id` se incluye por defecto.

Sintaxis:

```
{
  $project: {
    field1: 1,
    field2: 0,
    newField: { <expression> }
  }
}
```

Ejemplo:

```
{
  $project: {
    name: 1,
    totalWithIVA: { $multiply: ["$price", 1.21] }
  }
}
```

Operadores en agregación

Booleanos

- `$and`, `$or`, `$not`

Comparación

- `$eq`, `$ne`
- `$gt`, `$gte`
- `$lt`, `$lte`
- `$in`
- `$cmp` (0 iguales, 1 mayor, -1 menor)

Ejemplo:

```
{ $gt: ["$age", 18] }
```

Operadores sobre sets (arrays simples)

- `$setEquals`
- `$setIntersection`
- `$setUnion`
- `$setDifference`
- `$setIsSubset`
- `$anyElementTrue`
- `$allElementsTrue`

Operadores aritméticos

- `$add`
- `$subtract`
- `$multiply`
- `$divide`
- `$mod`

Ejemplo:

```
{ $add: ["$price", "$tax"] }
```

Operadores sobre cadenas

- `$concat`
- `$substr`
- `$toLower`
- `$toUpper`
- `$strcasecmp`

Ejemplo:

```
{ $concat: ["$name", " ", "$surname"] }
```

Operadores sobre listas y variables

- `$size`: tamaño del array
- `$map`: transforma cada elemento
- `$let`: define variables temporales
- `$literal`: devuelve un valor sin evaluar

Ejemplo:

```
{
  $map: {
    input: "$scores",
    as: "s",
    in: { $multiply: ["$$s", 2] }
  }
}
```

Operadores sobre fechas

- `$year`, `$month`, `$week`
- `$dayOfYear`, `$dayOfMonth`, `$dayOfWeek`
- `$hour`, `$minute`, `$second`, `$millisecond`

Ejemplo:

```
{ $year: "$createdAt" }
```

Operadores condicionales

- `$cond`: if–then–else
- `$ifNull`: valor por defecto si es null

Ejemplo:

```
{
  $cond: {
    if: { $gt: ["$age", 18] },
    then: "adult",
    else: "minor"
  }
}
```